QFabric: Guarded Cross-Domain Task Recovery and Its Limits on a Linux–RTOS Board

Ayushman Bhattacharya
Pollinations AI
ayushman.elixpo@gmail.com

Anwesha Chakraborty
Dept. CSE, JIS University
anwesha.elixpo@gmail.com

Abstract*—Heterogeneous embedded boards pair a Linux application processor with a real-time microcontroller (MCU), which suggests moving a task between the two domains when its timing degrades. We present QFabric, a prototype that treats cross-domain placement as guarded recovery from a violated empirical soft real-time contract: a bounded task is compiled for both domains, monitored in windows with hysteresis, switched only after semantic and capacity admission at a quiescent epoch boundary, and committed or rolled back after probation. On an Arduino UNO Q (Cortex-A53 Linux and a Cortex-M33 running Zephyr), we characterize the stock Bridge RPC interconnect and measure five workloads in both domains under idle, CPU-load, and higher-priority `SCHED_FIFO` contention, with ordinary and real-time callers. Each MCU invocation costs about 19 ms of dispatch regardless of compute, the MCU runs the heaviest kernel 34× slower, and because every call traverses Linux userspace, Linux contention inflates the MCU path far more than the Linux path (p99 up to 400 ms versus 17.0 ms). Across 30 cells of a four-node chain, Linux meets every contract within 0.8% misses and QFabric correctly never migrates; a controller without hysteresis migrates in 7 cells, raising p95 by up to 177×, and probation rolls back 5 of those moves. On Linux-dispatched heterogeneous boards, migration therefore cannot isolate a task from Linux interference, and recovery guards matter chiefly by preventing harmful moves. This paper supersedes an earlier artifact release whose reported improvement arose from measurement defects that we identify. All source, raw traces, and generators are public.

***Index Terms*—heterogeneous embedded systems, asymmetric multiprocessing, Linux, RTOS, interconnect characterization, timing contracts, runtime adaptation**

I. Introduction

Embedded platforms increasingly integrate two execution environments on one board: a Linux application processor for libraries, networking, and developer productivity, and a real-time microcontroller for isolation and predictable execution [1], [2]. A task that can run in either domain faces a placement choice whose consequences extend beyond compute time. Crossing the boundary adds serialization and transport, contention reshapes tail latency, and a move may be illegal for stateful or Linux-specific operations.

A natural idea is to treat the microcontroller as a refuge: when a task's timing degrades on Linux, move it to the real-time domain. Classical heterogeneous scheduling maps task graphs statically from cost estimates [3]; feedback scheduling adapts resources to measured error [4]; and runtime verification makes timing violations observable [5], [6]. What remains unclear on asymmetric Linux–RTOS boards is whether such a move can actually help, and how a runtime should decide when a faster prediction is not sufficient permission to migrate.

We study this question with QFabric, a deliberately narrow prototype on the Arduino UNO Q. A portable task declaration is compiled for both domains, profiled in each, and governed by an empirical contract. A recovery controller applies hysteresis to complete observation windows, admits a candidate only if semantic, capacity, and predicted-cost conditions hold, drains in-flight work at an epoch boundary, and commits the move only if post-switch probation confirms the contract; otherwise it rolls back.

This paper makes the following contributions:

- a **taxonomy** of cross-domain placement approaches by binding time, trigger, and switch verification, which locates guarded recovery relative to static mapping, partitioning, and mixed-criticality mode changes (Section III);
- a **guarded recovery protocol** combining a bounded wire ABI, effect-based semantic admission, quiescent epoch switching, and probation-based commit or rollback (Section IV);
- an **interconnect characterization** of the UNO Q Linux–MCU Bridge across payload size, concurrency, load, and frequency governor (Section VII-A);
- a **matched-load measurement study** of five workloads in both domains under three load conditions and two caller policies, with a per-call decomposition into compute and residual dispatch cost (Section VII-B); and
- an **evaluation of controller decisions** on a four-node chain, including an ablation showing that hysteresis and probation prevent harmful migrations (Sections VII-C–VII-D).

The central finding is a boundary, not a speedup. On this board the MCU path is dominated by a fixed dispatch cost of about 19 ms, and because every MCU invocation is issued and relayed by Linux processes, Linux interference degrades the MCU path more than the Linux path. Recovery by migration therefore never pays off in our measurements, and QFabric's guards earn their place by refusing moves that a naive controller would make. An earlier artifact release [7] reported a 36.6% recovery improvement; Section VI-F explains why that result does not hold.

The rest of the paper is organized as follows. Section II describes the platform and system model, Section III reviews related work through the taxonomy, Section IV presents

the design, Section V the implementation, Section VI the methodology, Section VII the results, and Section VIII discusses implications and threats to validity before Section IX concludes.

II. Background and System Model

A. Target platform

The Arduino UNO Q combines a quad-core Arm Cortex-A53 application processor running Debian Linux with an STM32U585 Cortex-M33 microcontroller running Zephyr [1]. Arduino's Bridge stack exposes MessagePack remote procedure calls in both directions through a Linux-resident router process [8]. The arrangement resembles remote-processor systems built on `remoteproc` and `rpmsg` [9], [10], but we use the stock UNO Q stack rather than modifying the transport. Fig. 1 shows how QFabric sits on this platform: the policy, the dispatcher, and the Bridge router all execute in Linux userspace, so every MCU invocation begins and ends there.

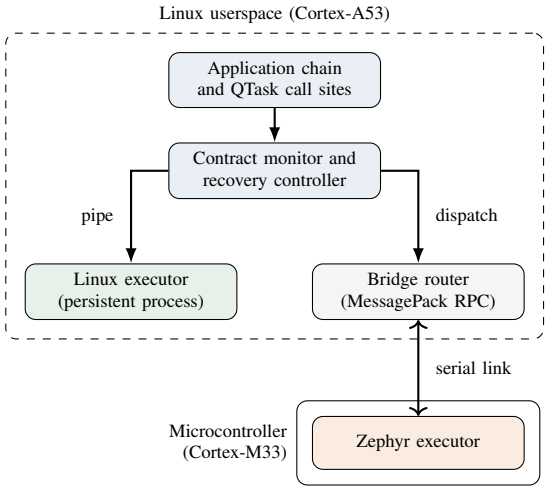


Fig. 1. QFabric on the UNO Q. The controller, dispatcher, and Bridge router run in Linux userspace; each task has a native implementation in both domains.

B. System model

An application is a chain $G = (V, E)$ of tasks. Each task $v \in V$ may have a Linux implementation, an MCU implementation, or both, together with fixed-width request and response types, a numeric identifier, and an effect class. Only pure, bounded tasks are migration eligible. A placement assigns a domain $d(v) \in \{\mathrm{L}, \mathrm{M}\}$ to every task.

Every invocation is issued from Linux userspace, so the latency observed for task v in domain d decomposes into local compute and a residual that captures dispatch, transport, and scheduling delay:

$$L_{v,d} = C_{v,d} + R_{v,d}. \tag{1}$$

For the MCU, R includes MessagePack encoding, the Linux router, the serial link, and the return path; for Linux it is the cost of a pipe round trip to a persistent executor. A chain observation costs $L_G = \sum_{v \in V} L_{v,d(v)}$, and the controller predicts it from profiled 95th percentiles:

$$\widehat{C}_G(d) = \sum_{v \in V} q_{.95}(v, d(v)). \tag{2}$$

This is a transparent heuristic, not a worst-case execution-time bound. Because each MCU sample already includes its round trip from Linux, no separate boundary term is added.

For a window w of n invocations with outcomes o_i and latencies L_i, the observed miss rate against a deadline D is

$$\widehat{m}_w = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[o_i \neq \mathrm{ok} \ \vee \ L_i > D], \tag{3}$$

and the window violates the contract when $\widehat{m}_w > \epsilon$. The contract is an empirical soft real-time objective.

C. Research questions

We ask: **RQ1**, what does the Linux–MCU interconnect cost, and how does that cost respond to payload, concurrency, load, and frequency policy? **RQ2**, under matched load conditions, how do the two domains compare per call, and where does the time go? **RQ3**, on a four-node chain, do contract violations arise, and does the controller decide correctly against deployable baselines and an oracle? **RQ4**, are hysteresis, admission, and probation necessary? **RQ5**, what does the mechanism itself cost?

III. Related Work and Taxonomy

We organize prior work along three dimensions that determine how a system reacts when timing degrades on a heterogeneous platform: *binding time* (when the assignment of software to execution resources is fixed), *trigger* (what causes a reassignment), and *switch verification* (whether a reassignment must prove its benefit before it is kept). Table I applies this taxonomy to representative work. A mark means the property is central to the cited work, not that it could not be added.

A. Static and feedback-driven placement

HEFT ranks a task graph and maps it to heterogeneous processors using estimated computation and communication costs [3]. It is the natural static reference but does not monitor deployed timing. Real-time scheduling theory provides admission and schedulability foundations [19], and feedback scheduling adjusts resource allocation from runtime error [4]; autonomic computing generalizes the monitor–analyze–plan–execute loop [20]. QFabric uses such a loop, but its actuator is a guarded change of execution domain.

B. Timing contracts and runtime verification

Runtime verification observes real-time behavior with bounded intrusion [5]. Stochastic contracts provide empirical runtime checks for component systems [6], while formal timing contracts and zone-based models support verification and synthesis [21], [22]. These works motivate our contract semantics; our focus is the operational protocol after a violation.

TABLE I
TAXONOMY OF CROSS-DOMAIN PLACEMENT APPROACHES. "SEMANTIC GUARD" MEANS EFFECT- OR STATE-BASED MIGRATION ELIGIBILITY; "VERIFIED SWITCH" MEANS AN OBSERVED POST-SWITCH CHECK DECIDES WHETHER THE MOVE IS KEPT.

Work	Binding time	Trigger	Isolation unit	Semantic guard	RT admission	Verified switch	Linux–RTOS board
HEFT [3]	design time	cost model	processor	–	–	–	–
Feedback scheduling [4]	runtime	control error	resource share	–	model-specific	–	–
Stochastic contracts [6]	runtime	contract monitor	component	–	–	–	–
Mode changes [11], [12]	runtime	mode or budget overrun	task set	–	✓	–	–
RT task migration [13]	runtime	resource management	core	–	✓	–	–
CoMOS [14]	runtime	runtime system	processor	–	–	–	–
YASMIN [15]	runtime	scheduler	processor	–	–	–	–
Awkernel [16]	runtime	DAG scheduler	core	–	–	–	–
Jailhouse / Bao [17], [18]	configuration	–	partition	–	–	–	–
OpenAMP / rpmsg [2], [10]	configuration	–	core	–	–	–	✓
QFabric	runtime	contract violation	task	✓	✓	✓	✓

C. Asymmetric multiprocessing and partitioning

Heterogeneous SoCs that pair application cores with real-time cores are commonly run as asymmetric multiprocessing systems, with Linux managing a remote RTOS through remoteproc and rpmsg; OpenAMP standardizes these interactions [2] and has been evaluated on real-time embedded SoC platforms [23]. Measurements on Zynq UltraScale+ show that inter-processor signaling can reach sub-microsecond latency when shared memory is used directly instead of the OpenAMP stack [24]. An alternative to physically separate cores is a static partitioning hypervisor such as Jailhouse [17] or Bao [18]; their real-time behavior and interference on Arm platforms have been compared empirically [25], and hypervisor-based isolation has been combined with programmable-logic memory on heterogeneous MPSoCs [26]. In these approaches the assignment of software to cores or partitions is fixed at configuration time. QFabric takes the Linux–RTOS split and its transport as given, and changes only which side executes a declared task after a contract violation.

D. Timing analysis and mixed criticality

QFabric's contracts are empirical soft real-time bounds, not WCET guarantees. Static and measurement-based WCET analysis [27], and measurement-based probabilistic timing analysis in particular [28], [29], give principled ways to derive bounds from observed executions, under assumptions that our percentile thresholds do not establish. The mixed-criticality literature, started by Vestal's model of tasks with several execution-time estimates of different assurance [12] and surveyed by Burns and Davis [30], also studies runtime adaptation when a lower-assurance estimate is exceeded. That reaction is usually a criticality mode change, a well-studied class of protocol [11]. QFabric's recovery can be read as a narrow, cross-domain instance of this pattern: a violated empirical bound triggers a guarded, verified change of placement, not a change of criticality mode, and we make no claim of certifiable guarantees.

E. Migration across heterogeneous processors

Real-time task migration has been studied for dynamic resource management in many-core systems with feasibility analysis [13], and Linux supplies deadline-aware scheduling mechanisms [31]. CoMOS provides processor-agnostic tasks and runtime migration across heterogeneous processors [14]; YASMIN supports functionally equivalent task implementations and user-space real-time mapping on heterogeneous COTS platforms [15]; and Awkernel brings DAG scheduling into an RTOS evaluated on physical hardware [16]. QFabric differs in its trigger (an observed empirical-contract violation), its guards (semantic eligibility and capacity admission), and its insistence on post-switch verification. To the best of our search, this combination on a physical Linux–RTOS boundary is new; we claim no novelty for placement, profiling, feedback control, or timing contracts individually. Our measurements also bound when the combination can help, which prior work on migration typically does not report.

IV. QFABRIC DESIGN

A. QTask declaration and bounded ABI

A QTask declaration is the single source for Linux and MCU artifacts. The generator accepts fixed-size scalars, records, and bounded arrays; it rejects pointers, dynamic arrays, recursive types, unsupported effects, and messages larger than the 988-byte payload budget. Versioned big-endian headers carry a magic value, message kind, task identifier, request identifier, payload length, and flags. Golden vectors are checked independently in Python, C++, and on the MCU, which turns serialization bounds and effect eligibility into auditable facts rather than conventions.

B. Contracts and admission

Profiles exclude warm-up invocations and retain raw observations; a minimum sample count marks an estimator valid. The per-task deadline is fixed by rule before any replay, from the idle profiles of both domains with ordinary callers:

$$D = \left\lceil 1.25 \max_{d \in \{\mathrm{L},\mathrm{M}\}} p99^{\mathrm{idle}}(v,d) \right\rceil_{1\,\mathrm{ms}}, \qquad D_G = |V|\, D. \tag{4}$$

Taking the slower domain makes the contract feasible for both alternatives at idle, so a violation reflects degradation rather than an infeasible target. A candidate MCU move must also leave utilization below a reserved headroom h:

$$U'_{\mathrm{M}} = U_{\mathrm{M}} + r_v\, q_{.95}(v,\mathrm{M}) \le 1 - h, \tag{5}$$

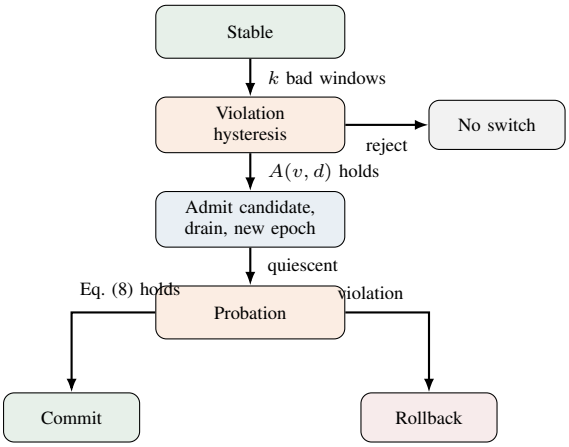


Fig. 2. Guarded recovery. Every terminal outcome returns the monitor to Stable.

where r_v is the invocation rate. A switch is admissible only when

$$A(v,d) = E_{\min} \wedge S_{\text{semantic}} \wedge S_{\text{admission}} \wedge \left(\widehat{C}_G(d) \le D_G\right) \wedge S_{\text{cooldown}}, \tag{6}$$

and the controller reports a stable rejection code for every failed conjunct.

C. Guarded recovery protocol

Fig. 2 shows the recovery state machine. With windows of $n = 20$ invocations and tolerance $\epsilon = 1\%$, analysis begins only after k consecutive violating windows:

$$\text{analyze} \iff \bigwedge_{j=0}^{k-1} \widehat{m}_{w-j} > \epsilon, \qquad k = 3. \tag{7}$$

If no candidate satisfies Eq. (6), placement is unchanged. Otherwise the dispatcher stops accepting work, waits until no invocation is in flight, increments a routing epoch, and changes the active domain. The move is committed only if the pooled miss rate over $P = 6$ probation windows meets the contract,

$$\text{commit} \iff \widehat{m}_{\text{prob}} \le \epsilon, \tag{8}$$

and is rolled back to the previous epoch and domain otherwise. A prediction thus authorizes probation, never commitment.

V. Implementation

The implementation comprises a Python controller and evidence pipeline, generated C++ ABI code, an AArch64 Linux executor, and Arduino/Zephyr sketches. The Linux executor is a persistent process that receives requests over a pipe and reports its own compute time with a monotonic clock. The MCU executor answers each request in a single Bridge call that returns both the result and the compute time measured with the Zephyr cycle counter, so no diagnostic calls fall on the measured path. Requests carry a 64-byte input block encoded in base64 (at most 96 characters), inside the 128-byte payload envelope that Section VII-A verifies; both executors share one decoder so that they perform identical work. Every result is checked against an independent Python reference.

Instrumentation is layered. Reduced mode records core timing, full mode adds structured observations, and disabled mode measures the base call path. An optional kernel probe is evaluated separately and is not required for control. An LED-matrix status display has disabled, static, and 10/30/60-Hz refresh profiles so that its cost is measured rather than assumed. All records carry schema versions, run identifiers, monotonic timestamps, outcomes, and mode metadata, and command-line audits cross-check source hashes, sample counts, derived deadlines, and reported numbers.

VI. Evaluation Methodology

A. Testbed and workloads

Table II summarizes the setup and Fig. 3 shows the board. All measurements use one UNO Q with its stock Bridge stack. The five workloads span a trivial task and a compute sweep: a signed 32-bit addition; a bitwise CRC-32 over the 64-byte block; and a 16-tap triangular Q15 FIR filter over 32 samples repeated 1, 64, or 512 times with a pass-dependent perturbation, written FIR×1, FIR×64, and FIR×512.

B. Load conditions and callers

Three conditions are applied for the whole duration of a run. *Idle* applies no load. *CPU load* runs eight `stress-ng` CPU workers under `SCHED_OTHER`. *RT contention* runs four `stress-ng` workers at `SCHED_FIFO` priority 80 with a 50% duty cycle in 10-ms slices, modeling a co-located higher-priority real-time workload that tuning the measured task cannot preempt. Under each condition both domains are profiled with an ordinary caller and with the caller and Linux executor at `SCHED_FIFO` priority 50, giving 12 profile runs and 120,000 retained invocations.

C. Evidence classes

We keep three evidence classes separate. *Direct measurements* are the per-call latencies above. *Hardware scenarios* exercise the real controller on the board and demonstrate one committed switch and one rollback. *Ordered trace replay* composes the measured traces into a four-node chain: each task visit consumes the next sample of the relevant trace in measured order, which preserves autocorrelation and regime changes, and 500 chain observations are replayed per cell. Replay runs the protocol of Section IV exactly, including Eqs. (4)–(8). It permits paired comparison across policies but is not equivalent to running a complete application.

D. Baselines

Table III lists the policies. The oracle chooses, for every task visit, the faster of the two domains' next measured samples; it is a non-causal lower bound, not a competitor.

TABLE II
EXPERIMENTAL SETUP

Item	Configuration
Board	Arduino UNO Q
Linux domain	Cortex-A53, Debian, AArch64
MCU domain	STM32U585 Cortex-M33, Zephyr
Transport	Bridge RPC / MessagePack
Workloads	add, CRC-32, FIR$\times\{1, 64, 512\}$
Samples per run	2000 after 100 warm-up
Load conditions	idle, CPU, RT contention
Caller policies	`SCHED_OTHER`, `SCHED_FIFO` 50
Per-task deadline	24–58 ms, Eq. (4)
Claim class	Empirical soft real-time

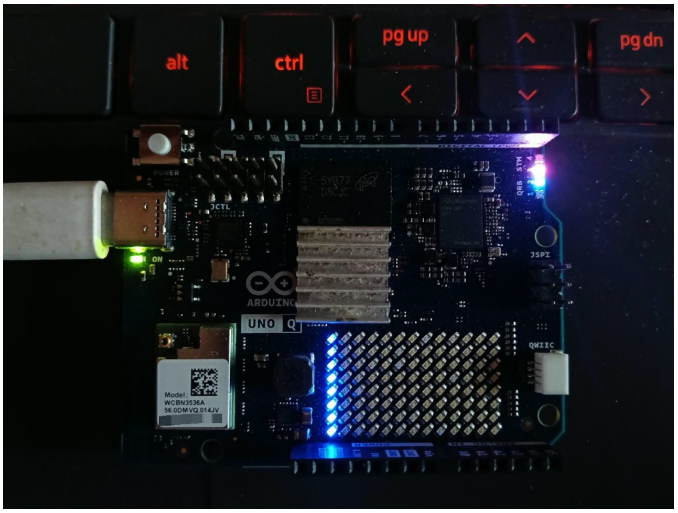


Fig. 3. Physical testbed. The LED matrix shows the controller's state for demonstration; it does not participate in decisions.

E. Statistics

Primary metrics are p95 latency and deadline-miss rate. For per-call p95 we report 95% confidence intervals from a circular block bootstrap (2,000 resamples, block length 50), which respects the short-range autocorrelation of measured traces. All tables, figures, and numbers in the text are generated from the raw traces by scripts in the artifact, which record input digests.

F. Relation to the earlier release

An earlier artifact release [7] included a manuscript whose flagship result was a 36.6% p95 reduction from recovery on a replayed chain under Linux load. Re-examination found two defects. First, its Linux executor was a new process per invocation, so Linux latency was dominated by process creation: 8.05 ms p95 for an addition, against 0.075 ms with the persistent executor used here. Second, the replay drew MCU latencies from idle traces while drawing Linux latencies from loaded traces, although every MCU call is dispatched through Linux and is affected by the same load. The present evaluation removes both defects and fixes its deadline rule and replay protocol in the repository before any new data was collected. Under the corrected method the earlier improvement does not hold. The earlier release remains available unchanged for transparency.

TABLE III
POLICIES COMPARED ON THE FOUR-NODE CHAIN

Policy	Role
Ordinary Linux	All tasks in Linux, default scheduling
Tuned Linux	All tasks in Linux, `SCHED_FIFO` caller
MCU-first	All eligible tasks on the MCU
Manual expert	Declared placement L, M, M, L
QFabric static	Domain with lower idle p95
QFabric recovery	Guarded recovery, ordinary caller
QFabric recovery (tuned)	Guarded recovery, `SCHED_FIFO` caller
Offline oracle	Non-causal per-visit minimum

VII. RESULTS

A. RQ1: Interconnect characterization

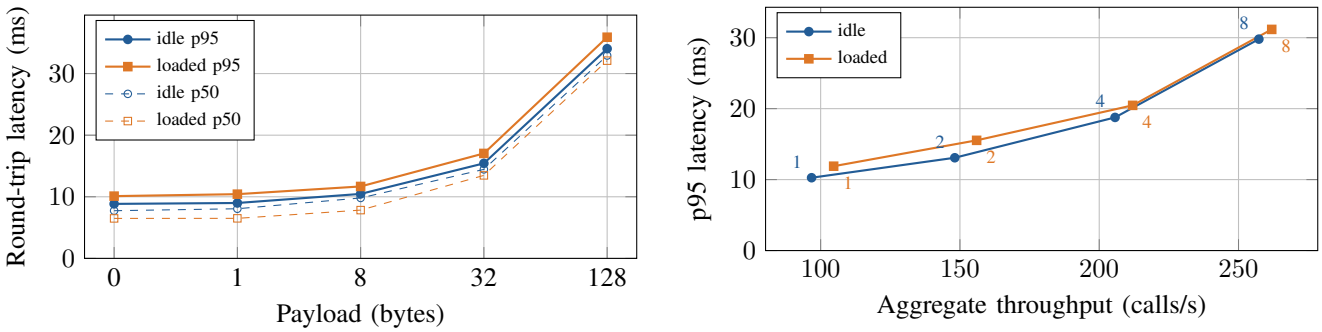
All cross-domain costs in QFabric originate from the Bridge RPC path between Linux and the MCU, so we characterized it before deriving any contract. Each point is the mean of three repetition-level percentiles over 1,000 Linux-observed round trips, recomputed from 108,000 raw samples with zero failures. Under the default `schedutil` governor, an idle zero-byte echo has a p50 of 7.74 ms and p95 of 8.85 ms, sustaining 127.2 sequential calls/s; at the largest demonstrated-safe payload, 128 bytes, p95 rises to 34.04 ms and the rate falls to 30.2 calls/s (Fig. 4a); latency grows nearly linearly with payload size. A 256-byte probe caused sustained loss of MCU RPC service, so we make no claim beyond 128 bytes.

Four `stress-ng` CPU workers lowered several medians but raised p95 by 5.5%–23.5% across Linux-initiated calls. Pinning the `performance` governor did not remove this tail penalty (6.7%–42.9%); it changed echo p95 by -11.1% to -1.3% at idle and by -0.2% to $+10.6\%$ under load. Frequency policy therefore does not explain the loaded tail. With concurrent 8-byte callers, aggregate throughput grows sublinearly from 96.75 to 257.31 calls/s (2.66×) at one to eight workers, while p95 rises from 10.27 to 29.80 ms (Fig. 4b). Load increased throughput by 1.8%–8.2% yet produced a 115.82-ms outlier, so contracts must be derived from tails, not throughput or medians.

B. RQ2: Matched-load per-call comparison

Table IV and Fig. 5 compare the domains per call. At idle, an MCU addition takes 18.81 ms at p95 against 0.075 ms in Linux, a 251× gap. Table V decomposes the idle median with Eq. (1). The MCU residual stays near 19 ms for every workload, so dispatch, not compute, dominates the MCU path; and the MCU computes FIR×512 34× more slowly than the Cortex-A53, so heavier kernels widen the gap rather than closing it.

Load changes the picture in an instructive way. CPU load raises the ordinary Linux p95 to a few milliseconds, which a `SCHED_FIFO` caller removes almost entirely. Under RT



(a) Payload sweep, one sequential caller

(b) 1/2/4/8 concurrent workers, 8-byte payload

Fig. 4. Linux↔MCU Bridge RPC characterization under the `schedutil` governor. Values are means of three repetition-level percentiles; "loaded" runs four `stress-ng` CPU workers. Point labels in (b) give the worker count.

TABLE IV
PER-CALL P95 LATENCY (MS) BY LOAD AND CALLER POLICY. BRACKETS GIVE 95% BLOCK-BOOTSTRAP CONFIDENCE INTERVALS FOR THE ORDINARY CALLER. "FIFO" DENOTES A `SCHED_FIFO` PRIORITY-50 CALLER.

		Linux p95 (ms)		MCU p95 (ms)	
Load	Workload	Ordinary [95% CI]	FIFO	Ordinary [95% CI]	FIFO
Idle	add	0.07 [0.07, 0.08]	0.06	18.81 [18.79, 18.84]	18.99
	CRC-32	0.15 [0.12, 0.16]	0.07	19.78 [19.76, 19.79]	19.36
	FIR×1	0.08 [0.08, 0.09]	0.07	20.16 [20.13, 20.19]	19.64
	FIR×64	0.45 [0.44, 0.46]	0.44	23.07 [23.00, 23.14]	22.88
	FIR×512	1.31 [1.22, 1.33]	1.18	45.93 [45.88, 46.02]	45.62
CPU load	add	0.18 [0.14, 0.23]	0.11	21.49 [21.16, 21.86]	17.01
	CRC-32	5.37 [5.22, 5.55]	0.17	23.17 [22.72, 23.35]	18.30
	FIR×1	5.13 [4.94, 5.41]	0.17	23.22 [22.80, 23.51]	18.52
	FIR×64	7.20 [6.56, 7.46]	0.45	27.42 [27.27, 27.75]	21.81
	FIR×512	9.24 [9.11, 9.39]	1.16	50.76 [50.31, 51.09]	45.07
RT contention	add	0.14 [0.10, 0.17]	0.08	39.99 [38.74, 42.70]	38.66
	CRC-32	0.24 [0.19, 0.27]	0.13	40.33 [38.48, 47.21]	35.19
	FIR×1	0.23 [0.17, 0.26]	0.17	42.04 [39.24, 44.47]	38.41
	FIR×64	0.76 [0.74, 0.78]	0.77	37.70 [36.36, 39.10]	38.96
	FIR×512	1.59 [1.54, 1.67]	1.45	200.37 [195.34, 219.40]	219.64

TABLE V
IDLE PER-CALL DECOMPOSITION (EQ. 1). COMPUTE IS MEASURED INSIDE EACH EXECUTOR; THE RESIDUAL IS THE DIFFERENCE OF MEDIANS.

	Compute p50 (μs)		Residual p50 (ms)	
Workload	Linux	MCU	Linux	MCU
add	0.1	0.8	0.053	18.53
CRC-32	1.3	26.9	0.060	19.09
FIR×1	1.7	52.4	0.060	19.27
FIR×64	95.7	3231.2	0.277	18.85
FIR×512	760.4	25842.1	0.380	19.12

contention, the Linux path keeps a p95 below 2 ms with occasional stalls, while the MCU path degrades sharply, with p99 up to 400 ms against 17.0 ms for Linux. An MCU invocation traverses the caller, the Bridge router, and serial-driver activity, all on Linux, so it is exposed to the co-runner at several points, whereas a Linux invocation needs one runnable core at one point. Moving a task to the MCU does not move its dispatch path off Linux.

C. RQ3: Chain-level outcomes and controller decisions

Table VI reports the replayed four-node chain. Linux meets the contract in every cell: the worst ordinary-Linux p95 is 26.7 ms against graph deadlines of at least 96 ms, and the worst miss rate of either Linux policy is 0.8%. MCU-first is several times slower in every cell. The offline oracle coincides with Linux except for rare visits in which a stalled Linux sample exceeds the paired MCU sample. With the hysteresis of Eq. (7), QFabric performs 0 switches across all 30 cells, which is the correct decision: no admissible alternative would have improved the contract.

D. RQ4: Are the guards necessary?

Two kinds of evidence address the guards. On hardware, a controlled scenario detected a sustained violation one complete window after onset with no false switch, switched to a feasible alternate, and verified it after four windows; a separate injected scenario forced probation failure and produced a rollback. A negative suite exercised nine stable rejection reasons: insufficient evidence, unsatisfied contract, semantic ineligibility, destination admission failure, MCU

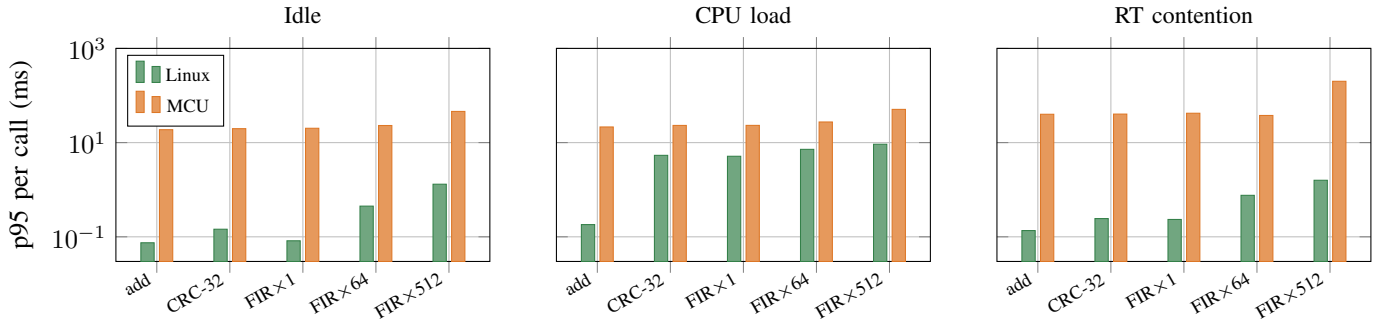


Fig. 5. Per-call p95 latency with ordinary callers, logarithmic scale. Linux contention moves the MCU path more than the Linux path because each MCU call is issued and relayed by Linux processes.

TABLE VI
REPLAYED FOUR-NODE CHAIN (500 OBSERVATIONS PER CELL). D_G IS THE GRAPH DEADLINE OF EQ. (4); "STAY" MEANS NEITHER RECOVERY VARIANT SWITCHED.

			Ordinary Linux		Tuned Linux		MCU-first	QFabric
Load	Workload	D_G (ms)	p95	miss	p95	miss	p95	decision
Idle	add	96	0.27	0.0%	0.22	0.0%	74.6	stay
	CRC-32	100	0.52	0.0%	0.26	0.0%	78.5	stay
	FIR×1	104	0.32	0.0%	0.25	0.0%	78.6	stay
	FIR×64	120	1.69	0.0%	1.63	0.0%	90.5	stay
	FIR×512	232	4.99	0.0%	4.65	0.0%	181.9	stay
CPU load	add	96	5.34	0.0%	0.40	0.0%	76.5	stay
	CRC-32	100	9.34	0.0%	0.64	0.0%	83.0	stay
	FIR×1	104	8.81	0.0%	0.66	0.0%	82.0	stay
	FIR×64	120	17.31	0.0%	1.69	0.0%	101.3	stay
	FIR×512	232	26.70	0.0%	4.52	0.0%	191.3	stay
RT contention	add	96	0.58	0.2%	0.31	0.0%	444.5	stay
	CRC-32	100	0.85	0.2%	0.56	0.0%	458.9	stay
	FIR×1	104	0.81	0.2%	0.71	0.2%	452.8	stay
	FIR×64	120	3.97	0.8%	2.97	0.2%	253.6	stay
	FIR×512	232	20.07	0.0%	17.00	0.8%	539.3	stay

headroom, predicted deadline miss, higher predicted end-to-end cost, observed miss-rate failure, and active cooldown. These scenarios demonstrate the protocol's mechanics; their latencies are not used as performance evidence.

Table VII isolates hysteresis on the Stage 12 traces by setting $k = 1$ in Eq. (7). The ablated controller reacts to isolated Linux stalls under RT contention and switches in 7 cells, raising p95 by up to 177× and increasing miss rates. Probation, Eq. (8), rolls back 5 of these moves; the remaining two were still in probation when the trace ended ("open at end"). Hysteresis prevents the harmful moves, and probation bounds the damage when a move is made anyway.

TABLE VII
HYSTERESIS ABLATION UNDER RT CONTENTION: SWITCHING ON THE FIRST VIOLATING WINDOW ($k = 1$) INSTEAD OF $k = 3$.

Workload	Caller	Win.	p95 (ms)	Miss (%)	Outcome
add	ordinary	16	0.58→103.32	0.2→6.4	rolled back
CRC-32	ordinary	21	0.85→95.05	0.2→3.8	open at end
FIR×1	ordinary	15	0.81→102.03	0.2→4.8	rolled back
FIR×1	FIFO	7	0.71→102.89	0.2→4.4	rolled back
FIR×64	ordinary	4	3.97→109.51	0.8→3.8	rolled back
FIR×64	FIFO	21	2.97→101.46	0.2→0.8	open at end
FIR×512	FIFO	9	17.00→379.48	0.8→10.4	rolled back

E. RQ5: Mechanism overhead

Table VIII reports the cost of the mechanism itself. One policy decision costs 170.1 μs of CPU time and 177.2 μs of wall time at p95 over 1,000 in-process evaluations, with a 3138-byte peak observed by `tracemalloc`; the latter excludes the interpreter baseline and is not total process memory. A decision is thus two orders of magnitude cheaper than one MCU dispatch. The worst LED-matrix refresh profile changes Bridge p95 by 2.05%.

TABLE VIII
MEASURED MECHANISM COSTS

Component	Result
Policy decision CPU time, p95	170.1 μs
Policy decision wall time, p95	177.2 μs
Policy traced peak allocation	3138 bytes
Linux dispatch, add, idle p95	0.075 ms
MCU dispatch, add, idle p95	18.81 ms
Worst LED display delta	2.05%

TABLE IX
ANSWERS TO THE RESEARCH QUESTIONS

RQ	Answer
RQ1	About 9 ms p95 per zero-byte call, rising with payload; tails grow under load regardless of governor.
RQ2	Linux is faster in every cell; MCU dispatch is a fixed ≈19 ms and Linux contention hurts the MCU path more.
RQ3	No contract fails on Linux; QFabric correctly makes 0 switches in 30 cells.
RQ4	Yes: without hysteresis, 7 harmful switches; probation rolls back 5.
RQ5	A decision costs about 177.2 μs, negligible against dispatch.

VIII. DISCUSSION

A. When can cross-domain recovery pay off?

Equation (1) makes the boundary explicit. Migrating task v from Linux to the MCU helps only if $C_{v,\mathrm{M}} + R_{v,\mathrm{M}} < C_{v,\mathrm{L}} + R_{v,\mathrm{L}}$ under the degraded condition. On this board $R_{v,\mathrm{M}} \approx 19$ ms at idle, $C_{v,\mathrm{M}}$ exceeds $C_{v,\mathrm{L}}$ by more than an order of magnitude, and the interference that inflates $R_{v,\mathrm{L}}$ inflates $R_{v,\mathrm{M}}$ at least as much, because both residuals are paid in Linux userspace. The inequality therefore fails for any task dispatched one call at a time from Linux. Three architectural changes could reverse it: a lower-latency transport such as shared-memory signaling, which reaches sub-microsecond latency on other heterogeneous SoCs [24]; a dispatcher outside Linux userspace, for example a kernel driver or a partitioned core [25]; or MCU-resident chains whose release is triggered by an MCU timer so that only inputs and results cross the boundary. Each is a direct test of the boundary reported here.

B. What the guards contribute

Recovery and tuning solve different problems. When the workload can remain in Linux, a `SCHED_FIFO` caller removes most CPU-load interference and is the right first step. The value of QFabric's guards is that they make adaptive placement safe to leave enabled: hysteresis ignores isolated stalls, admission rejects moves predicted to miss, and probation reverses moves whose benefit does not materialize. Table VII shows that a plausible but unguarded controller would have turned sub-millisecond tails into 100-ms tails.

C. Programmer effort

A task declaration states types, effect, and identifier once, and the generated ABI removes hand-written marshaling. The programmer must still supply semantically equivalent implementations for both domains, choose a defensible contract, and state Linux-only effects; QFabric shifts effort from transport plumbing to contracts and equivalence rather than removing it.

D. Threats to validity

External validity. Results come from one UNO Q, one software stack, and five small kernels. Boards with faster transports or dispatch outside Linux userspace may reverse the inequality above. The chain results replay measured traces rather than running a full application.

Construct validity. Percentiles and miss rates do not establish worst-case execution time or hard real-time guarantees, and the admission test of Eq. (5) is a utilization gate, not a schedulability proof. The residual in Table V is a difference of medians, not a measured component.

Internal validity. All runs share one board and may be affected by firmware, thermal state, and background activity. Ordered replay preserves temporal structure within a trace but cannot represent interactions between tasks executing concurrently. The deadline rule and replay protocol were committed to the repository before any Stage 12 data was collected.

Baseline and load validity. `SCHED_FIFO` is a strong but not exhaustive Linux baseline; `SCHED_DEADLINE` [31], CPU isolation, and affinity warrant comparison. The RT-contention co-runner is one synthetic pattern; synchronized or fully saturating interference would change magnitudes, though it would burden the MCU dispatch path as well.

Safety scope. Quiescent switching prevents mixed-epoch calls in the prototype and effect annotations reject known-ineligible tasks, but this is not a proof of implementation equivalence. Production use would need state transfer, authentication, fault containment, and multi-client analysis.

IX. CONCLUSION

QFabric demonstrates a guarded form of runtime adaptation across a Linux–RTOS boundary: an empirical contract violation can trigger a quiescent, admitted switch whose benefit must be observed before it is kept. Measured on an Arduino UNO Q with matched load conditions, the Linux–MCU path costs about 19 ms per call, the MCU computes up to 34× more slowly, and Linux interference reaches the MCU path through its Linux-resident dispatch. Migration consequently never improves a contract in our study, the controller correctly declines to migrate, and an unguarded controller would have made harmful moves that only probation undoes. The broader lesson for heterogeneous embedded architectures is that the isolation offered by a real-time core is available only to work whose dispatch also leaves the general-purpose domain. Testing faster transports, non-Linux dispatch, and MCU-resident chains is the natural next step.

Artifact Availability

The repository [32] contains source, generated code, unit tests, board sketches, raw and processed traces, campaign logs, audits, and the generators for every number, table, and figure in this paper; running `make` in the `paper` directory regenerates the assets and builds the PDF. The release cited here, `v1.1.0`, is archived on Zenodo with DOI `10.5281/zenodo.23692354`.

References

[1] Arduino, *UNO Q User Manual*, Arduino, 2026, accessed 2026-10-09. [Online]. Available: https://docs.arduino.cc/tutorials/uno-q/user-manual/

[2] OpenAMP Project, "The OpenAMP project," Hosted by Linaro, 2026, accessed 2026-10-27. [Online]. Available: https://www.openampproject.org/

[3] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[4] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Feedback control real-time scheduling: Framework, modeling, and algorithms," *Real-Time Systems*, vol. 23, pp. 85–126, 2002.

[5] T. Reinbacher, M. Függer, and J. Brauer, "Runtime verification of embedded real-time systems," *Formal Methods in System Design*, vol. 44, pp. 203–239, 2014.

[6] P. R. Nandi, A. Monot, and M. Oriol, "Stochastic contracts for runtime checking of component-based real-time systems," in *Proceedings of the 18th International ACM SIGSOFT Symposium on Component-Based Software Engineering*, 2015, pp. 138–147.

[7] A. Bhattacharya and A. Chakraborty, "QFabric: Contract-driven cross-domain task recovery," GitHub release v1.0.1, 2026, earlier artifact release; its manuscript is superseded by this paper. [Online]. Available: https://github.com/elixpo/qfabric/releases/tag/v1.0.1

[8] Arduino, *Arduino RouterBridge Library*, Arduino, 2026, accessed 2026-10-09. [Online]. Available: https://github.com/arduino-libraries/Arduino_RouterBridge

[9] Linux Kernel Developers, *Remote Processor Framework*, The Linux Kernel Documentation, 2026, accessed 2026-10-09. [Online]. Available: https://docs.kernel.org/staging/remoteproc.html

[10] ——, *Remote Processor Messaging (rpmsg) Framework*, The Linux Kernel Documentation, 2024, accessed 2026-10-09. [Online]. Available: https://www.kernel.org/doc/html/v6.8/staging/rpmsg.html

[11] J. Real and A. Crespo, "Mode change protocols for real-time systems: A survey and a new proposal," *Real-Time Systems*, vol. 26, no. 2, pp. 161–197, 2004.

[12] S. Vestal, "Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance," in *Proceedings of the 28th IEEE International Real-Time Systems Symposium (RTSS)*, 2007, pp. 239–243.

[13] B. Pourmohseni, A. Smirnov, S. Wildermann, and J. Teich, "Real-time task migration for dynamic resource management in many-core systems," in *Next Generation Real-Time Embedded Systems*, ser. OpenAccess Series in Informatics, vol. 77, 2020, pp. 5:1–5:14.

[14] M. Goraczko, C.-C. Han, J. Helander, J. Liu, B. Priyantha, and F. Zhao, "CoMOS: An operating system for heterogeneous multi-processor sensor devices," Microsoft Research, Tech. Rep. MSR-TR-2006-177, 2006.

[15] B. Rouxel, S. Altmeyer, and C. Grelck, "YASMIN: A real-time middleware for COTS heterogeneous platforms," in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 298–309.

[16] A. Yano, R. Nishimura, Y. Kobayashi, Y. Takano, T. Ishikawa-Aso, and T. Azumi, "Awkernel: RTOS bridging DAG scheduling theory and component-oriented real-time systems practice," 2026.

[17] R. Ramsauer, J. Kiszka, D. Lohmann, and W. Mauerer, "Look mum, no VM exits! (almost)," in *Proceedings of the 13th Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPERT)*, 2017.

[18] J. Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto, "Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems," in *Next Generation Real-Time Embedded Systems*, ser. OpenAccess Series in Informatics, vol. 77, 2020, pp. 3:1–3:14.

[19] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[20] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[21] M. A. Khatib, A. Girard, and T. Dang, "Verification and synthesis of timing contracts for embedded controllers," in *Proceedings of the 19th International Conference on Hybrid Systems: Computation and Control*, 2016.

[22] M. Camilli, A. Gargantini, and P. Scandurra, "Zone-based formal specification and timing analysis of real-time self-adaptive systems," *Science of Computer Programming*, vol. 159, pp. 28–57, 2018.

[23] S. Alonso, J. Lazaro, J. Jimenez, L. Muguira, and U. Bidarte, "Evaluating the OpenAMP framework in real-time embedded SoC platforms," in *Proceedings of the 36th Conference on Design of Circuits and Integrated Systems (DCIS)*, 2021, pp. 1–6.

[24] N. Coppik, P. Becker, and M. Ritter, "Low-latency real-time applications on heterogeneous MPSoCs," in *Next Generation Real-Time Embedded Systems*, ser. OpenAccess Series in Informatics, vol. 128, 2025, pp. 2:1–2:14.

[25] J. Martins and S. Pinto, "Shedding light on static partitioning hypervisors for Arm-based mixed-criticality systems," in *Proceedings of the 29th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2023, pp. 40–53.

[26] G. Gracioli, R. Tabish, R. Mancuso, R. Mirosanlou, R. Pellizzoni, and M. Caccamo, "Designing mixed criticality applications on modern heterogeneous MPSoC platforms," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, ser. Leibniz International Proceedings in Informatics, vol. 133, 2019, pp. 27:1–27:25.

[27] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström, "The worst-case execution-time problem—overview of methods and survey of tools," *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–53, 2008.

[28] L. Cucu-Grosjean, L. Santinelli, M. Houston, C. Lo, T. Vardanega, L. Kosmidis, J. Abella, E. Mezzetti, E. Q. nones, and F. J. Cazorla, "Measurement-based probabilistic timing analysis for multi-path programs," in *Proceedings of the 24th Euromicro Conference on Real-Time Systems (ECRTS)*, 2012, pp. 91–101.

[29] R. I. Davis and L. Cucu-Grosjean, "A survey of probabilistic timing analysis techniques for real-time systems," *Leibniz Transactions on Embedded Systems*, vol. 6, no. 1, pp. 03:1–03:60, 2019.

[30] A. Burns and R. I. Davis, "A survey of research into mixed criticality systems," *ACM Computing Surveys*, vol. 50, no. 6, pp. 1–37, 2017.

[31] J. Lelli, C. Scordino, L. Abeni, and D. Faggioli, "Deadline scheduling in the Linux kernel," *Software: Practice and Experience*, vol. 46, no. 6, pp. 821–839, 2016.

[32] A. Bhattacharya and A. Chakraborty, "QFabric: Guarded cross-domain task recovery on a Linux–RTOS board," Zenodo, software release v1.1.0, 2026, source repository: https://github.com/elixpo/qfabric; Apache-2.0. [Online]. Available: https://doi.org/10.5281/zenodo.23692354